

RAG — Retrieval-Augmented Generation

Concise Study Notes

1. What is RAG?

- **RAG = Retrieval-Augmented Generation.**
- RAG allows an LLM to use external knowledge while generating an answer.
- Instead of relying only on the model's trained knowledge, RAG retrieves relevant information from a knowledge base.
- The retrieved information is provided to the LLM as **context**.

Basic Flow

User Query → Retrieve Relevant Information → Add Context → LLM → Final Answer

2. Why Do We Need RAG?

- LLM knowledge may become outdated.
- LLMs do not automatically know private or company documents.
- LLMs can hallucinate information.
- Updating an LLM through training can be expensive.
- RAG allows the knowledge base to be updated without retraining the LLM.

3. RAG Architecture

Indexing Pipeline: Documents → Loading → Chunking → Embeddings → Vector Database

Query Pipeline: User Query → Query Embedding → Retriever → Relevant Chunks → LLM → Answer

4. Main Components

Component	Purpose
Document Loader	Loads PDFs, DOCX, TXT, websites, etc.
Text Splitter	Divides large documents into smaller chunks.
Embedding Model	Converts text into numerical vectors.
Vector Database	Stores and searches vectors.
Retriever	Finds relevant chunks.
LLM	Generates the final answer.
Prompt	Provides instructions and retrieved context.

5. Document Loading

- The first step is to collect the data required by the RAG system.
- Common sources: PDF, DOCX, TXT, CSV, websites, databases and APIs.
Example: Company_Policy.pdf → Document Loader → Text

6. Text Splitting / Chunking

- Large documents are divided into smaller pieces called **chunks**.
- Chunks make retrieval more accurate and prevent sending the entire document to the LLM.
- Good chunking helps preserve useful context.

Splitter	Main Idea
CharacterTextSplitter	Splits text based on characters/separators.
RecursiveCharacterTextSplitter	Tries multiple separators and generally preserves structure better.
TokenTextSplitter	Splits text based on tokens.

Common choice: RecursiveCharacterTextSplitter for general-purpose text.

7. Embeddings

- An embedding is a numerical representation of text.
- Text is converted into a vector so its semantic meaning can be compared with other text.
- Similar meanings generally produce vectors that are closer to each other.
"What is MBA eligibility?" → Embedding Model → [0.21, -0.45, 0.73, ...]

8. Vector Database

- A vector database stores embeddings of document chunks.
- It allows the system to search for chunks that are semantically similar to a user's query.

Technology	Purpose
FAISS	Vector similarity search library
ChromaDB	Vector database
Pinecone	Managed vector database
Qdrant	Vector database
Weaviate	Vector database

9. Retrieval & Retriever

- **Retrieval** is the process of finding the most relevant chunks for a query.
- A **Retriever** performs the search and returns relevant information.
- The retriever does not generate the final answer; the LLM does that.
Query → Retriever → Relevant Chunks → LLM

10. Similarity Search

- The query is converted into an embedding.
- The query vector is compared with stored document vectors.
- The most similar vectors are selected.

Method	Purpose
Cosine Similarity	Measures similarity based on vector direction.
Euclidean Distance	Measures distance between vectors.
Dot Product	Measures vector similarity.

11. RAG + LLM

User Query + Retrieved Context → LLM → Final Answer

- Retrieved information is provided to the LLM as context.
- The LLM uses the context to generate the response.
- A good prompt can instruct the LLM to answer only from the provided context.

12. RAG vs Traditional LLM

Traditional LLM	RAG
Uses trained knowledge	Uses external knowledge
Knowledge can become outdated	Knowledge base can be updated
No automatic access to private documents	Can use private documents
No retrieval step	Has a retrieval step

13. RAG vs Fine-Tuning

RAG	Fine-Tuning
Retrieves external information	Trains the model further
Knowledge stored externally	Knowledge/behavior learned by model
Easy to update documents	Updating requires additional training
Good for document-based Q&A	Good for changing model behavior/style

Simple Rule: Need external/private knowledge → RAG. Need to change model behavior → Fine-Tuning.

14. Important RAG Terms

Term	Meaning
Chunk	Small part of a document.
Embedding	Numerical representation of text.
Vector Database	Stores and searches vectors.
Retriever	Finds relevant information.
Context	Retrieved information given to the LLM.
Top-K	Number of results retrieved.
Metadata	Additional information about a document.
Reranking	Reorders retrieved results by relevance.
Grounding	Generating answers based on retrieved information.
Hallucination	Incorrect or unsupported generated information.

15. Types of RAG

Type	Description
Naive RAG	Basic retrieve → generate pipeline.
Advanced RAG	Adds techniques such as reranking, filtering and query improvement.
Agentic RAG	An agent decides when and how to retrieve information.

16. Agentic RAG

- Agentic RAG combines RAG with an agent.
- The agent can decide whether retrieval is needed and which tool/retriever to use.
- This makes the workflow more flexible than a fixed RAG pipeline.

User Query → Agent → Decide/Use Retriever → Information → LLM → Final Answer

17. Common RAG Problems

- **Poor Chunking:** Important information may be split incorrectly.
- **Poor Retrieval:** Irrelevant chunks are retrieved.
- **Too Much Context:** Unnecessary information can confuse the LLM.
- **Hallucination:** The LLM may still generate unsupported information.
- **Outdated Data:** Old documents can produce outdated answers.
- **Duplicate Chunks:** Repeated information can reduce retrieval quality.

18. Improving RAG

- Use better chunking strategies.
- Choose a suitable embedding model.
- Tune the Top-K value.
- Use metadata filtering.
- Use hybrid search (keyword + vector search).
- Use reranking.
- Improve the RAG prompt.
- Regularly update the knowledge base.
- Evaluate retrieval and answer quality.

19. RAG Applications

- PDF Question Answering
- Company Knowledge Bases
- Customer Support Chatbots
- Educational Assistants
- Legal Document Search
- Product Search
- Code Documentation
- Enterprise Search

20. RAG in One View

Documents → Chunking → Embeddings → Vector Database → Retriever → Relevant Context → LLM → Final Answer

Remember: RAG = Retrieve + Augment + Generate